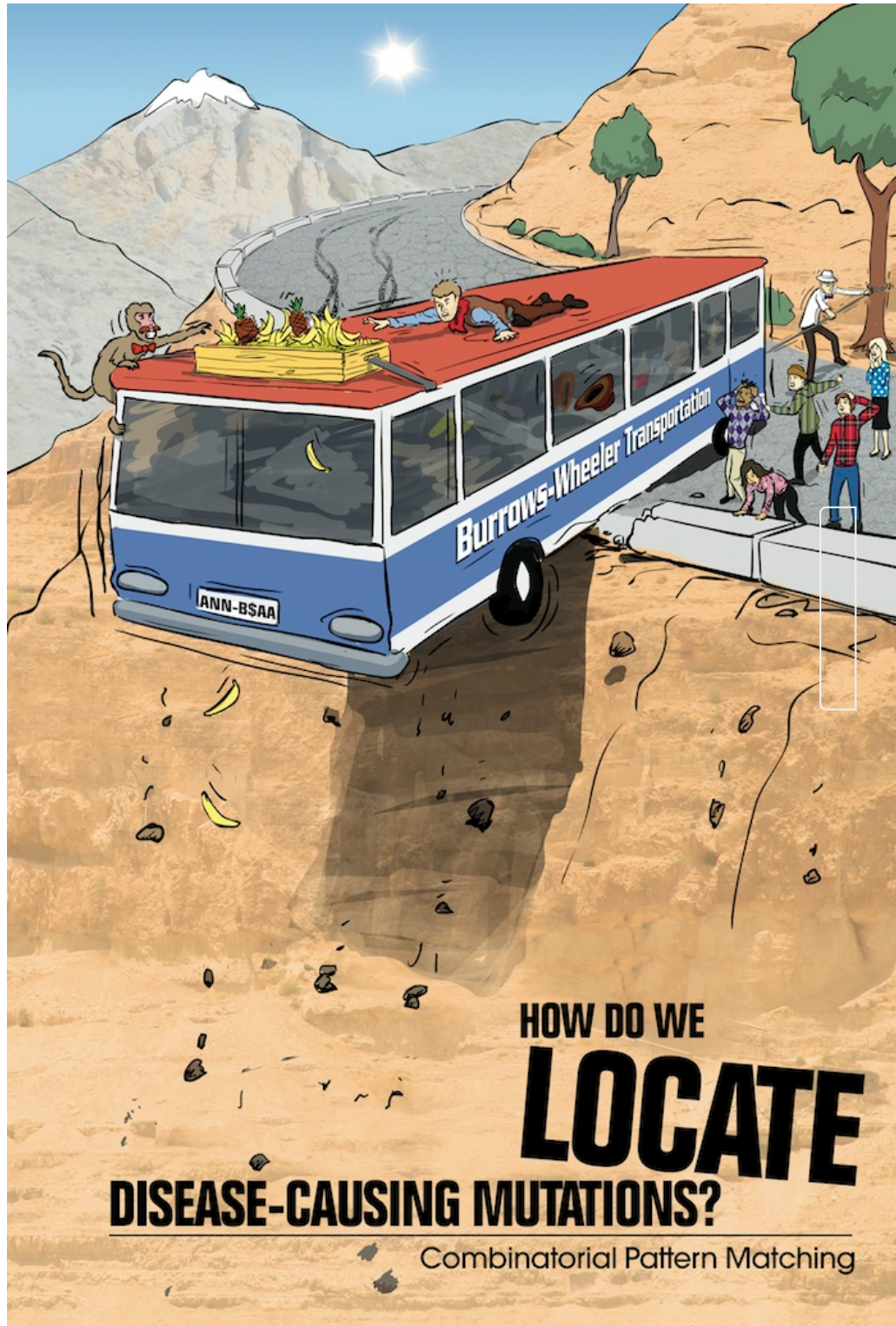






About 1% of babies are born with mental retardation, but this affliction remains poorly understood because it can be caused by a variety of different genetic disorders. One of these disorders is **Ohdo syndrome**, which causes an expressionless, “mask-like” face. In 2011, biologists solved the genetic puzzle underlying Ohdo syndrome by discovering a handful of mutations shared by multiple patients, which the researchers used to identify a single protein-truncating mutation responsible for Ohdo syndrome.

The discovery of Ohdo syndrome’s root cause represents just one of many new discoveries arising from the use of **read mapping** to study genetic disorders. In read mapping, researchers compare sequenced DNA reads taken from an individual against a **reference human genome** (see [DETOUR: The Reference Human Genome \(/lesson/lesson-305/\)](/lesson/lesson-305/)) in order to find which reads match the reference and which reads indicate mutations of one nucleotide into another (**single nucleotide polymorphisms**, or **SNPs**). The reference genome is a gross simplification of species identity, since in addition to about 3 million SNPs (0.1% of the human genome), humans differ by genome rearrangements, insertions, and deletions that can span thousands of nucleotides (see [DETOUR: Rearrangements, Insertions, and Deletions in Human Genomes \(/lesson/lesson-306/\)](/lesson/lesson-306/)). However, in this chapter we will focus only on algorithms for finding SNPs.



But wait, you may say, why not use one of the algorithms that we already covered? After all, we could always assemble the entire genome of an individual and then compare it to the reference genome. However, sequencing methods are computationally intensive and not perfect, as they often generate error-prone contigs. As a result, it makes sense to map reads from an individual human to the reference human genome to find out the differences.

To see why read mapping should be easier than genome assembly, let us return to the analogy of a jigsaw puzzle, which is sold with a picture of the completed puzzle on its box. This photo makes reconstructing the puzzle far easier; for a simple example, if the completed puzzle shows a sun in a blue sky, then you can automatically move all of the bright yellow pieces and all of the light blue pieces to the top of the puzzle.

Yet aside from genome sequencing, two other methods come to mind for read mapping. First, you could align each read to the reference genome (using a fitting alignment) to find the most similar region. Second, you could apply approximate pattern matching algorithms to match each read one at a time against the reference genome.

STOP and Think: What computational challenges might arise from using these methods to map millions of reads to a reference human genome?



What Causes Ohdo Syndrome? | Step 4

0 / 0

Both of these methods are guaranteed to solve the problem of mapping reads to a reference genome, but their runtimes become a bottleneck when we scale to millions of reads. Therefore, our goal in this chapter is to figure out how to use the reference genome as a “photo on the box” shortcut to finding SNPs.





Recall from the chapter on genome assembly that reads are typically a few hundred base pairs long. These reads will form a collection of strings *Patterns* that we wish to match against a genome *Text*. For each string in *Patterns*, we will first find all its *exact* matches as a substring of *Text* (or conclude that it does not appear in *Text*). When hunting for the cause of a genetic disorder, we can immediately eliminate from consideration areas of the reference genome where exact matches occur. In the epilogue, we will generalize this problem to find *approximate* matches, where single nucleotide substitutions in reads separate the individual from the reference genome (or represent errors in reads).

Multiple Pattern Matching Problem: *Find all occurrences of a collection of patterns in a text.*

Input: A string *Text* and a collection *Patterns* containing (shorter) strings.

Output: All starting positions in *Text* where a string from *Patterns* appears as a substring.

A naive approach to the Multiple Pattern Matching Problem would attempt repeated applications of an algorithm for the (single) Pattern Matching Problem, which we encountered when looking for the origin of replication in bacterial genomes. This algorithm, which we call

BRUTEFORCEPATTERNMATCHING, would slide each *Pattern* along *Text*, checking whether the substring starting at each position of *Text* matches *Pattern*. Recall that the runtime of a naive algorithm for a single pattern is $O(|Text| \cdot |Pattern|)$. Therefore, the total runtime of **BRUTEFORCEPATTERNMATCHING** for the Multiple Pattern Matching Problem is $O(|Text| \cdot |Patterns|)$, where $|Text|$ is the length of *Text* and $|Patterns|$ is the sum of the lengths of all strings in *Patterns*.

The problem with applying **BRUTEFORCEPATTERNMATCHING** to read mapping is that $|Text|$ and $|Patterns|$ are both huge. In the case of the human genome (3 GB), the total length of all reads may exceed 1 TB; as a result, any algorithm with runtime $O(|Text| \cdot |Patterns|)$ will be too slow.

STOP and Think: The estimate $O(|Text| \cdot |Patterns|)$ presents the *worst-case* approximation of the runtime for **BRUTEFORCEPATTERNMATCHING**. What is the *average-case* estimate?



Constructing a trie

The reason why the runtime of **BRUTEFORCEPATTERNMATCHING** is so high is that each string in *Patterns* must traverse all of *Text* independently. If you think about *Text* as a long road, then **BRUTEFORCEPATTERNMATCHING** is analogous to strapping each pattern into a car, an inefficient strategy. Instead, our goal is to herd the patterns onto a bus so that we only need to make one trip from the beginning to the end of *Text*. In more formal terms, we would like to organize *Patterns* into a data structure to prevent multiple passes down *Text* and to reduce the runtime. To this end, we will consolidate *Patterns* into a directed tree called a **trie** (pronounced “try”), which is written *Trie(Patterns)* and has the following properties (see the figure on the next step).

- The trie has a single node with indegree 0, called the **root**.
- Each edge of *Trie(Patterns)* is labeled with a letter of the alphabet.
- Edges leading out of a given node have distinct labels.
- Every string in *Patterns* is spelled out by concatenating the letters along some path from the root downward.
- Every path from the root to a **leaf**, or node with outdegree 0, spells a string from *Patterns*.



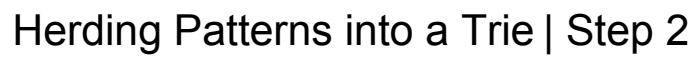


Figure: The trie for the following *Patterns*: "ananas", "and", "antenna", "banana", "bandana", "nab", "nana", "pan".



The most obvious way to construct $Trie(Patterns)$ is by iteratively adding each string from $Patterns$ to the growing trie, as implemented by the following algorithm.

TRIECONSTRUCTION($Patterns$)

$Trie \leftarrow$ a graph consisting of a single node $root$

for each string $Pattern$ in $Patterns$

$currentNode \leftarrow root$

for $i \leftarrow 1$ to $|Pattern|$

if there is an outgoing edge from $currentNode$ with label $currentSymbol$

$currentNode \leftarrow$ ending node of this edge

else

 add a new node $newNode$ to $Trie$

 add a new edge from $currentNode$ to $newNode$ with label $currentSymbol$

$currentNode \leftarrow newNode$

return $Trie$

Trie Construction Problem: Construct a trie on a set of patterns.

Input: A collection of strings $Patterns$.

Output: $Trie(Patterns)$.

STOP and Think: How can we use the trie to solve the Multiple Pattern Matching Problem?



CODE CHALLENGE: Solve the Trie Construction Problem.

Input: A collection of strings *Patterns*.

Output: The adjacency list corresponding to *Trie(Patterns)*, in the following format. If

Trie(Patterns) has n nodes, first label the root with 0 and then label the remaining nodes with the integers 1 through $n - 1$ in any order you like. Each edge of the adjacency list of *Trie(Patterns)* will be encoded by a triple: the first two members of the triple must be the integers labeling the initial and terminal nodes of the edge, respectively; the third member of the triple must be the symbol labeling the edge.

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradata/bowtie/TrieConstruction.txt\)](http://bioinformaticsalgorithms.com/data/extradata/bowtie/TrieConstruction.txt)

Sample Input:

```
ATAGA
ATC
GAT
```

Sample Output:

```
0->1:A
1->2:T
2->3:A
3->4:G
4->5:A
2->6:C
0->7:G
7->8:A
8->9:T
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-294/step/4



Applying the trie for multiple pattern matching

Given a string *Text* and *Trie(Patterns)*, we can quickly check whether any string from *Patterns* matches a *prefix* of *Text*. To do so, we start reading symbols from the beginning of *Text* and see what string these symbols “spell” as we proceed along the path downward from the root of the trie, as illustrated in the figure below. For each new symbol in *Text*, if we encounter this symbol along an edge leading down from the present node, then we continue along this edge; otherwise, we stop and conclude that no string in *Patterns* matches a prefix of *Text*. If we make it all the way to a leaf, then the pattern spelled out by this path matches a prefix of *Text*.

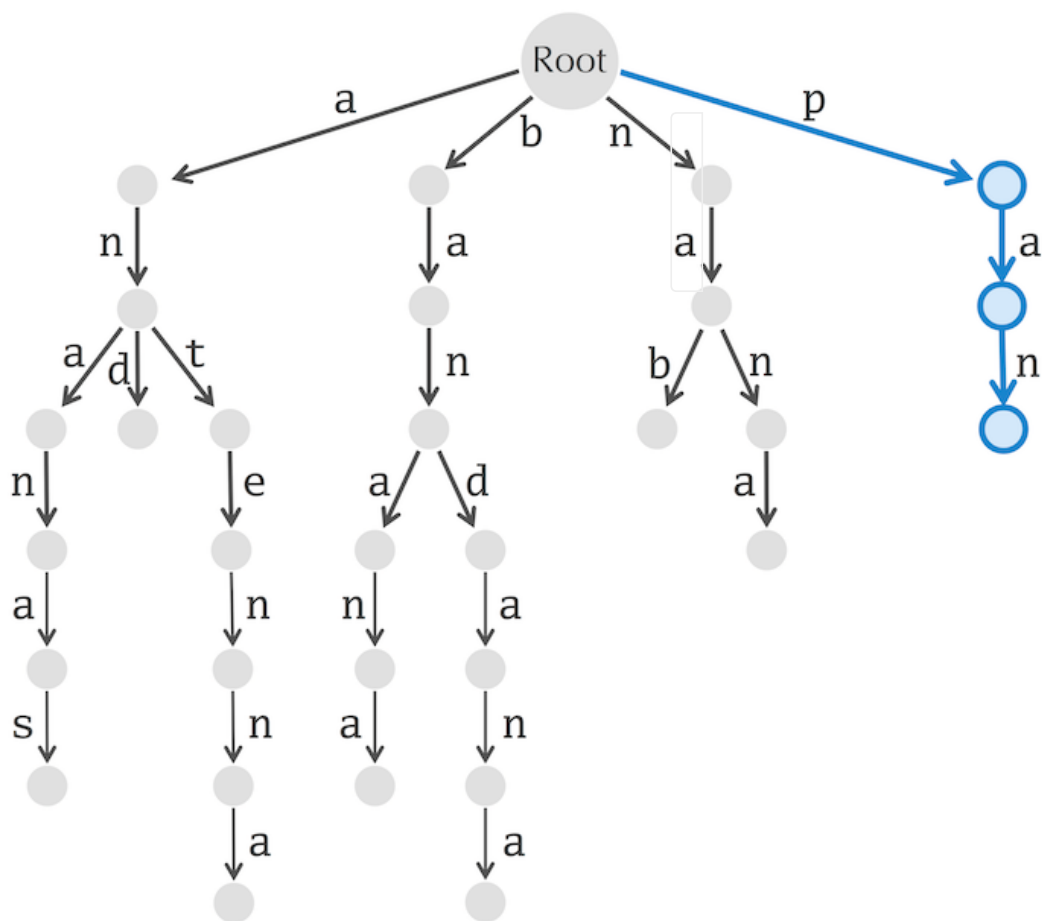


Figure: The pattern "pan" matches *Text* = "**pan**amabananas" when starting at the beginning of *Text*.



The algorithm described on the previous step is called **PREFIXTRIEMATCHING**.

PREFIXTRIEMATCHING(*Text*, *Trie*)

symbol $\leftarrow$ first letter of *Text*

v $\leftarrow$ root of *Trie*

while forever

if *v* is a leaf in *Trie*

return the pattern spelled by the path from the root to *v*

else if there is an edge (*v*, *w*) in *Trie* labeled by *symbol*

symbol $\leftarrow$ next letter of *Text*

v $\leftarrow$ *w*

else

output "no matches found"

return

STOP and Think: For **PREFIXTRIEMATCHING** to work, we have made a hidden assumption that no string in *Patterns* is a prefix of another string in *Patterns*. Why is this the case? Hint: consider adding "pantry" to the patterns in the trie from the previous step.



PREFIXTRIEMATCHING finds whether any strings in *Patterns* match a prefix of *Text*. To find whether any strings in *Patterns* match a substring of *Text* starting at position k , we chop off the first $k - 1$ symbols from *Text* and run **PREFIXTRIEMATCHING** on the shortened string. As a result, to solve the Multiple Pattern Matching Problem, we simply iterate **PREFIXTRIEMATCHING** $|Text|$ times, chopping the first symbol off of *Text* before each new iteration (see the figure below).

```
TRIEMATCHING(Text, Trie)  
  while Text is nonempty  
    PREFIXTRIEMATCHING(Text, Trie)  
    remove first symbol from Text
```

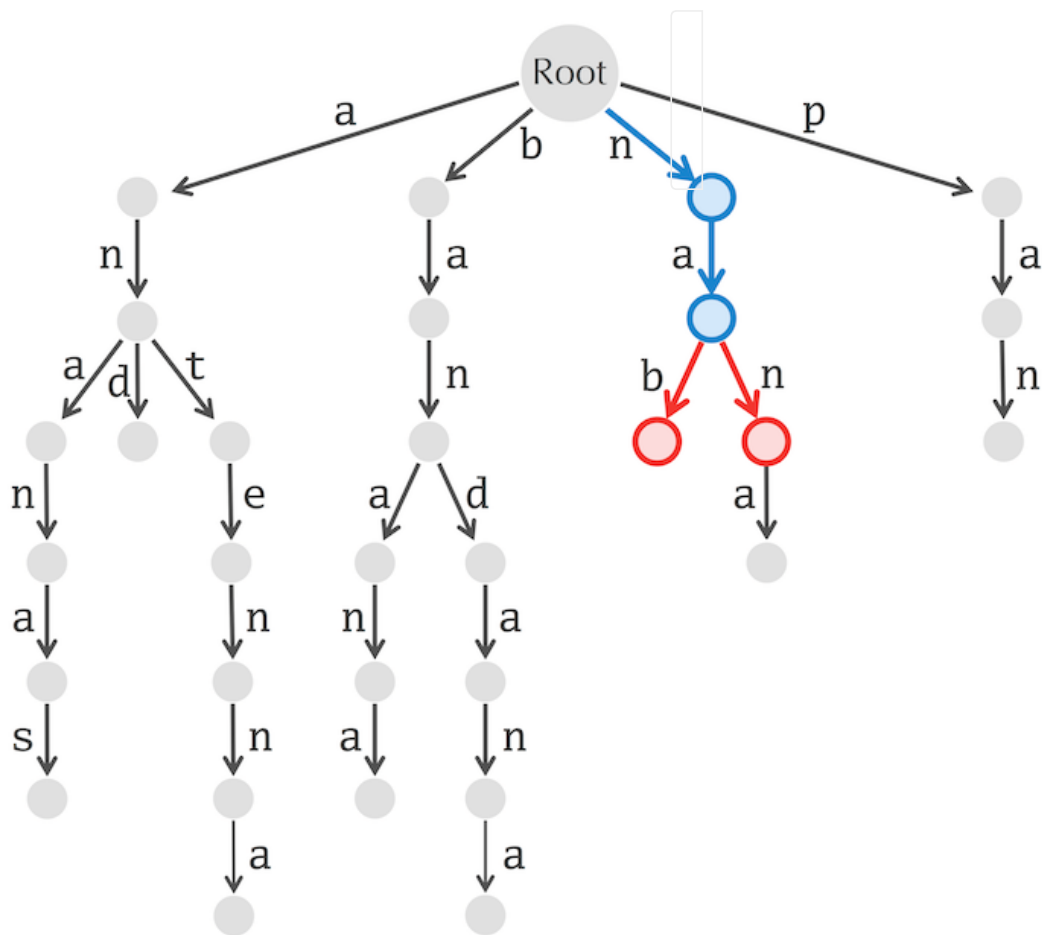


Figure: No pattern match is found in *Trie(Patterns)* when starting at the third symbol of *Text* = "panamabananas".



CODE CHALLENGE: Implement **TRIEMATCHING** to solve the Multiple Pattern Matching Problem.

Input: A string *Text* and a collection of strings *Patterns*.

Output: All starting positions in *Text* where a string from *Patterns* appears as a substring.

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/TrieMatching.txt\)](http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/TrieMatching.txt)

Sample Input:

```
AATCGGGTTCAATCGGGT
ATCG
GGGT
```

Sample Output:

```
1 4 11 15
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-294/step/8



We need need $|Patterns|$ steps to construct $Trie(Patterns)$, which contains at most $|Patterns|$ nodes. Each iteration of **PREFIXTRIEMATCHING** takes at most $|LongestPattern|$ steps, where $LongestPattern$ is the longest string in $Patterns$. **TRIEMATCHING** makes $|Text|$ total calls to **PREFIXTRIEMATCHING**, making the total number of steps equal to $|Patterns| + |Text| \cdot |LongestPattern|$. This runtime offers a huge speed-up compared to the $|Text| \cdot |Patterns|$ steps required by **BRUTEFORCEPATTERNMATCHING**. The Aho-Corasick algorithm, developed in 1975, further reduces the number of steps required after constructing the trie from $O(|Text| \cdot |LongestPattern|)$ steps to $O(|Text|)$ steps (see [DETOUR: The Aho-Corasick Algorithm \(/lesson/lesson-307/\)](/lesson/lesson-307/)).

STOP and Think: Do you see any computational challenges with using **TRIEMATCHING** to solve the Multiple Pattern Matching Problem?





Although **TRIEMATCHING** is fast, storing a trie consumes a lot of memory. Recall that **BRUTEFORCEPATTERNMATCHING** works with a single read at a time, which keeps the memory low because we only need to store the genome in memory. Yet **TRIEMATCHING** needs to store the entire trie in memory, which is proportional to $|Patterns|$. Since a collection of reads for the human genome may consume upwards of 1 TB, the memory required to store the trie is prohibitive.

STOP and Think: How can we avoid multiple passes through the genome without needing to consolidate all the reads into a huge data structure?





Introduction to suffix tries

Since storing $\text{Trie}(\text{Patterns})$ requires so much memory, let's process Text into a data structure instead. Our goal is to compare each string in Patterns against Text without needing to traverse the full length of Text . In more familiar terms, instead of packing Patterns onto a bus and riding the long distance down Text , our new data structure will be able to “teleport” each string in Patterns directly to its occurrences in Text .





A **suffix trie**, denoted $\text{SuffixTrie}(\text{Text})$, is the trie formed from all suffixes of Text (see figure below). From now on, we append the dollar-sign ("\$\$") to Text in order to mark the end of Text . We will also label each leaf of the resulting trie by the starting position of the suffix whose path through the trie ends at this leaf (using 0-based indexing). This way, when we arrive at a leaf, we will immediately know where this suffix came from in Text .

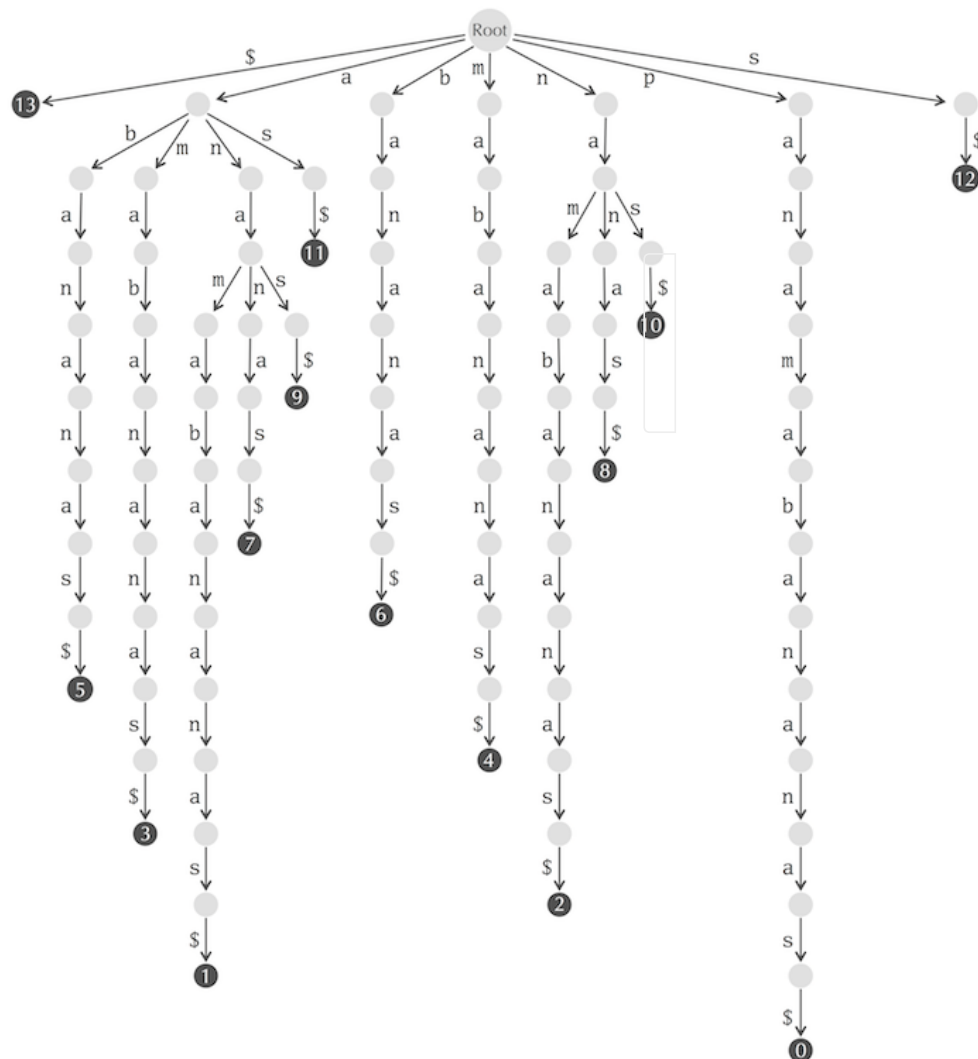


Figure: $\text{SuffixTrie}(\text{"panamabananas\$"})$, with leaf labels (corresponding to starting positions of suffixes) varying from 0 to 13.



Using suffix tries for pattern matching

To match a single string *Pattern* to *Text*, note that if *Pattern* matches a substring of *Text* starting at position *i*, then *Pattern* must also appear at the beginning of the suffix of *Text* starting at position *i*. We can therefore determine whether *Pattern* occurs in *SuffixTrie(Text)* by starting at the root and spelling out symbols of *Pattern* downward. If we can find a path in the suffix trie spelling out *Pattern*, then we know that *Pattern* must occur in *Text* (see figure below). We can then iterate over all strings in *Patterns*.

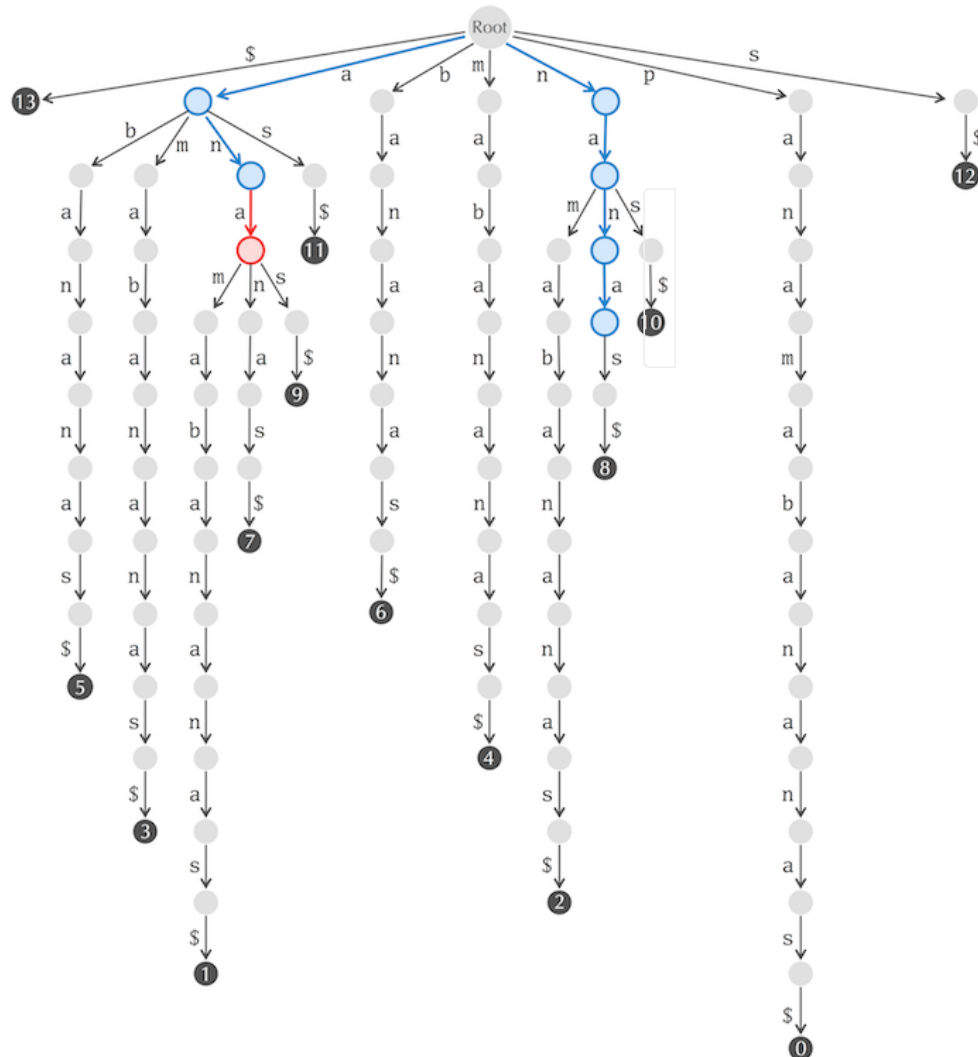
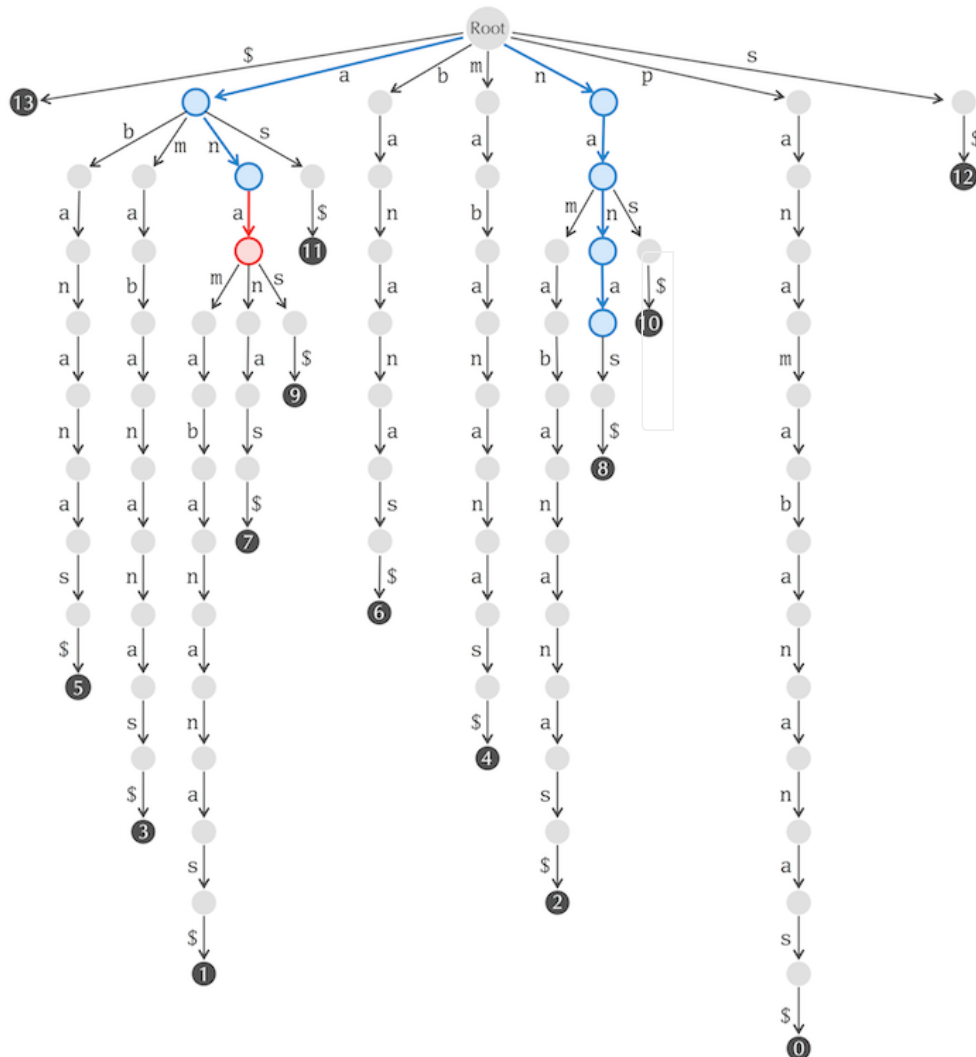


Figure: Threading "antenna" through *SuffixTrie*("panamabanananas\$") fails to find a match because no suffix of "panamabanananas\$" begins with "ant"; however, threading "nana" through the suffix trie does find a match: "panamabanananas\$".





To determine where *Pattern* appears in *Text*, assume first that *Pattern* matches *Text* at a leaf of *SuffixTrie(Text)*. In this case, *Pattern* must appear in *Text* as a suffix, and we can consult the label at that leaf to determine the starting position of the *suffix*. For example, threading "nana" into *SuffixTrie*("panamabananas\$"), reproduced below, shows that this pattern matches the suffix starting at position 8 of "panamabananas\$".





Preprocessing the Genome Instead | Step 7

0 / 0

STOP and Think: How much runtime and memory will it take to construct *SuffixTrie(Text)*?





Recall that constructing $Trie(Patterns)$ required $O(|Patterns|)$ runtime and memory. Accordingly, the runtime and memory required to construct $SuffixTrie(Text)$ are both equal to the combined length of all suffixes in $Text$. There are $|Text|$ suffixes of $Text$, ranging in length from 1 to $|Text|$ and having total length $|Text| \cdot (|Text| + 1) / 2$, which is $O(|Text|^2)$. Thus, we need to reduce both the construction time and memory requirements of suffix tries to make them practical.





Let's not give up hope on suffix tries. We can reduce the number of edges in $\text{SuffixTrie}(\text{Text})$ by combining the edges on any non-branching path into a single edge. We then label this edge with the concatenation of symbols on the consolidated edges, as shown in the figure below. The resulting data structure is called a **suffix tree**, written $\text{SuffixTree}(\text{Text})$.

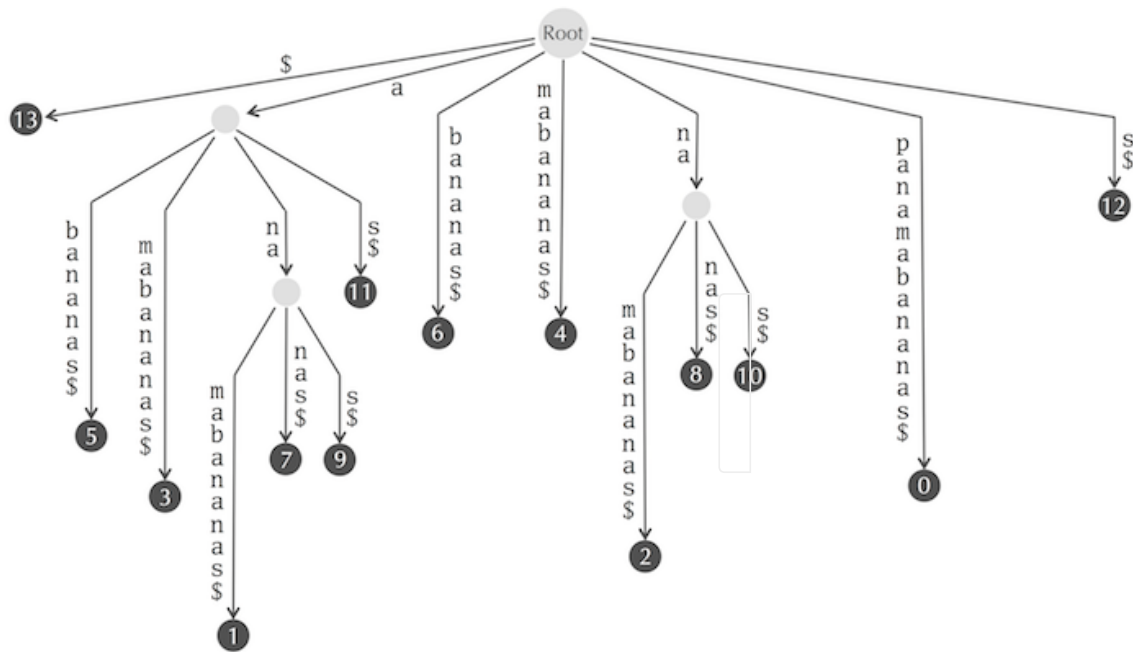


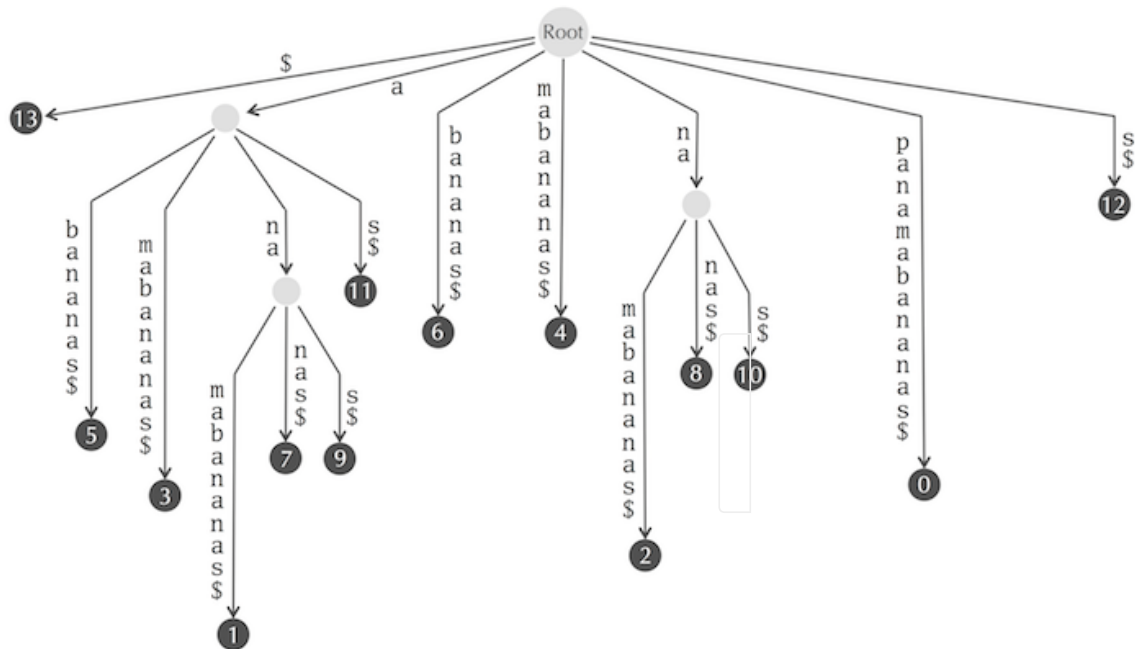
Figure: $\text{SuffixTree}(\text{"panamabananas\$"})$.

To match a single *Pattern* to *Text*, we thread *Pattern* into $\text{SuffixTree}(\text{Text})$ by the same process used for a suffix trie. Similarly to the suffix trie, we can use the leaf labels to find starting positions of successfully matched patterns.

EXERCISE BREAK: Prove that $\text{SuffixTree}(\text{Text})$ has exactly $|\text{Text}|$ leaves and at most $|\text{Text}|$ other nodes. (Ungraded exercise.)



Compared to a suffix trie, which may have a quadratic number of nodes in the length of *Text*, the number of nodes in *SuffixTree(Text)* is at most $2 \cdot |Text|$. Therefore, the memory required for *SuffixTree(Text)* is $O(|Text|)$.



STOP and Think: Wait a second. Suffix trees seem like a cosmetic modification of suffix tries. Since we still need to keep all concatenated edge labels in memory, why should suffix trees be so much more memory-efficient than suffix tries?



Suffix trees save memory because they do not need to store concatenated edge labels from each non-branching path. For example, a suffix tree does not need 10 bytes to store the edge labeled "mabananas\$" in *SuffixTree*("panamabananas\$") (reproduced below); instead, it suffices to store a pointer to position 4 of "panamabananas\$", as well as the *length* of "mabananas\$". Furthermore, suffix trees can be constructed in linear time, without having to first construct the suffix trie! We will not ask you to implement this fast suffix tree construction algorithm because it is quite complex.

Suffix Tree Construction Problem: *Construct the suffix tree of a string.*

Input: A string *Text*.

Output: *SuffixTree(Text)*.

A memory-inefficient way of constructing the suffix tree is to consolidate each non-branching path of the suffix trie into a single edge and to store its label in memory. To see how one might begin to implement a more memory-efficient solution, check out CHARGING STATION: Constructing a Suffix Tree (<https://stepic.org/lesson/Constructing-a-Suffix-Tree-9664>).



CODE CHALLENGE: Solve the Suffix Tree Construction Problem.

Input: A string *Text*.

Output: The edge labels of *SuffixTree(Text)*. You may return these strings in any order.

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/SuffixTreeConstruction.txt\)](http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/SuffixTreeConstruction.txt)

Sample Input:

```
ATAAATG$
```

Sample Output:

```
AAATG$
G$
T
ATG$
TG$
A
A
AAATG$
G$
T
G$
$
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-296/step/4



Although the suffix tree decreases memory requirements from $O(|Text|^2)$ to $O(|Text|)$, on average it still requires about 20 times as much memory as *Text*. In the case of a 3 GB human genome, 60 GB of RAM is a huge improvement over the 1 TB that we needed to work with *Trie(Patterns)*, but it still presents a memory challenge for most machines. This reveals a dark secret of Big-O notation, which is that it ignores constant factors. For huge datasets such as the human genome, we will need to pay attention to this constant factor, since the expression $O(|Text|)$ applies to both an algorithm with $2 \cdot |Text|$ memory and an algorithm with $1000 \cdot |Text|$ memory.

Yet before seeing how we can further reduce the memory needed for multiple pattern matching, we ask you to solve three problems showing how suffix trees can be applied to other pattern matching challenges. The first such problem is the Longest Repeat Problem.

Longest Repeat Problem: *Find the longest repeat in a string.*

Input: A string *Text*.

Output: A longest repeat in *Text*, i.e., a longest substring of *Text* that appears in *Text* more than once.

CODE CHALLENGE: Solve the Longest Repeat Problem. (Multiple solutions may exist, in which case you may return any one.)

Sample Input:

ATATCGTTTTATCGTT

Sample Output:

TATCGTT

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradata/bowtie/LongestRepeat.txt\)](http://bioinformaticsalgorithms.com/data/extradata/bowtie/LongestRepeat.txt)

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-296/step/5



The second additional exercise that we will consider is the Longest Shared Substring Problem.

Longest Shared Substring Problem: Find the longest substring shared by two strings.

Input: Strings $Text_1$ and $Text_2$.

Output: The longest substring that occurs in both $Text_1$ and $Text_2$.

CODE CHALLENGE: Solve the Longest Shared Substring Problem. (Multiple solutions may exist, in which case you may return any one.)

Sample Input:

TCGGTAGATTGCGCCCACTC
AGGGGCTCGCAGTGTAAGAA

Sample Output:

AGA

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/LongestSharedSubstring.txt\)](http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/LongestSharedSubstring.txt)

One way of solving the Longest Shared Substring Problem would be to construct two suffix trees, one for $Text_1$ and one for $Text_2$. Check out [CHARGING STATION: Solving the Longest Shared Substring Problem \(https://stepic.org/lesson/CS-Solving-the-Longest-Shared-Substring-Problem-9665\)](https://stepic.org/lesson/CS-Solving-the-Longest-Shared-Substring-Problem-9665) to learn about a more elegant solution.

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-296/step/6



Finally, we ask you to solve the Shortest Non-Shared Substring Problem.

Shortest Non-Shared Substring Problem: Find the shortest substring of one string that does not appear in another string.

Input: Strings $Text_1$ and $Text_2$.

Output: The shortest substring of $Text_1$ that does not appear in $Text_2$.

CODE CHALLENGE: Solve the Shortest Non-Shared Substring Problem. (Multiple solutions may exist, in which case you may return any one.)

Sample Input:

CCAAGCTGCTAGAGG

CATGCTGGGCTGGCT

Sample Output:

AA

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradata/bowtie/ShortestNonSharedSubstring.txt\)](http://bioinformaticsalgorithms.com/data/extradata/bowtie/ShortestNonSharedSubstring.txt)

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-296/step/7



To construct a suffix tree, we will first modify the construction of the suffix trie as follows. Although each edge *edge* in a suffix tree is labeled by a single symbol $Symbol(edge)$, it is unclear where this symbol came from in *Text*. We will therefore add another label for each edge (denoted $Position(edge)$) referring to the position of this symbol in *Text*. If $Symbol(edge)$ corresponds to more than one position in *Text*, then we will assign it its minimum starting position.

For example, consider the modified suffix trie for *Text* = "panamabananas", shown in the figure on the next step. There are five edges labeled by "m", colored purple and all labeled by position **4**, because this is the only occurrence of "m" in *Text*. On the other hand, the green edge corresponding to "n" is a different story. By following every path from this edge down to the leaves, we see that it corresponds to occurrences of "n" in the suffixes "anamabananas\$", "ananas\$", and "anas\$", at positions **2**, **8**, and **10**. As a result, we assign this edge the minimum of these positions.



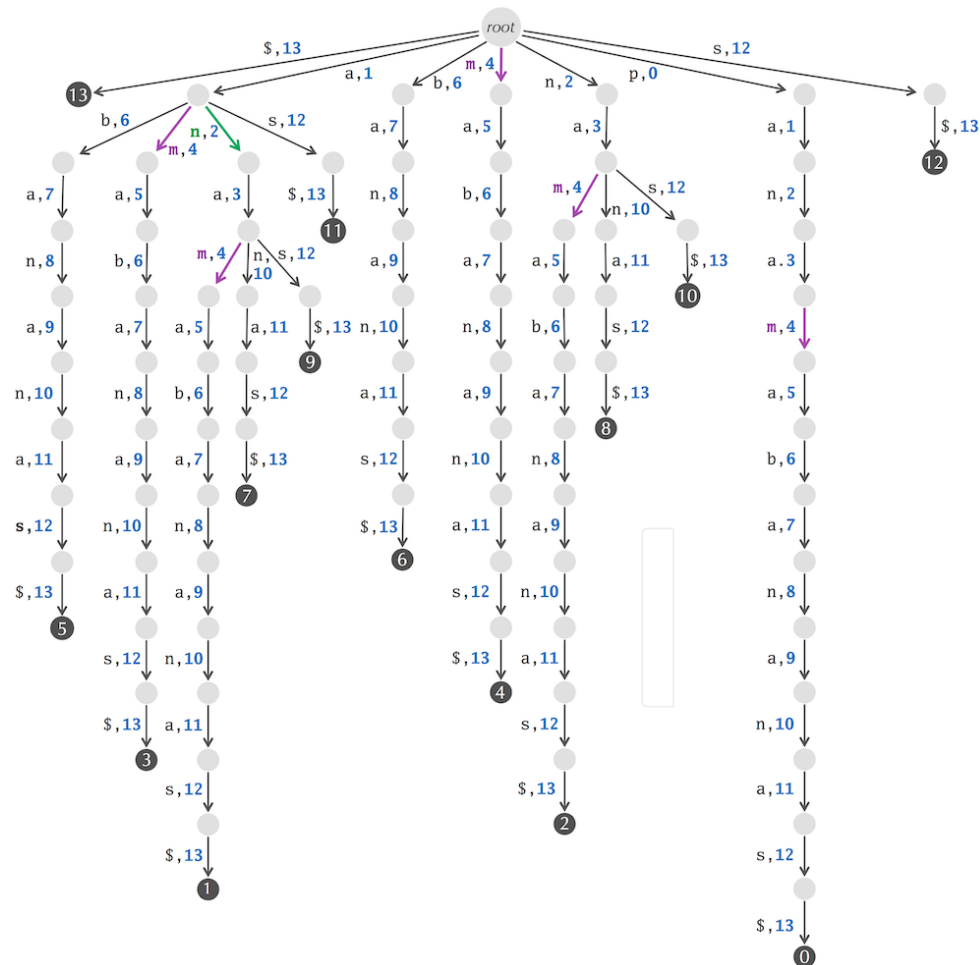
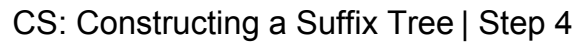


Figure: The modified suffix trie of $Text = \text{"panamabananas\$"}.$ Each edge is assigned the minimum position to which the edge's symbol corresponds in Text (shown in blue).

The following pseudocode constructs the modified suffix trie of a string *Text* by traversing the suffixes of *Text* from longest to shortest. Given a suffix, it attempts to spell the suffix downward by following edge labels as far as possible until it can go no further. At that point, it adds the rest of the suffix to the trie in the form of a path to a leaf, along with the position of each symbol in the suffix.

MODIFIEDSUFFIXTRIECONSTRUCTION(*Text*)

```
Trie  $\leftarrow$  a graph consisting of a single node root
for i  $\leftarrow$  0 to  $|Text| - 1$ 
    currentNode  $\leftarrow$  root
    for j  $\leftarrow$  i to  $|Text| - 1$ 
        currentSymbol  $\leftarrow$  j-th symbol of Text
        if there is an outgoing edge from currentNode labeled by currentSymbol
            currentNode  $\leftarrow$  ending node of this edge
        else
            add a new node newNode to Trie
            add an edge newEdge connecting currentNode to newNode in Trie
            Symbol(newEdge)  $\leftarrow$  currentSymbol
            Position(newEdge)  $\leftarrow$  j
            currentNode  $\leftarrow$  newNode
        if currentNode is a leaf in Trie
            assign label i to this leaf
return Trie
```



We can now transform a modified suffix trie into a suffix tree as follows. In the suffix tree from the main text, each edge $edge$ in $SuffixTree("panamabananas\$")$ is labeled by a string of symbols, denoted $String(edge)$. As we mentioned in the main text, storing all these strings is memory-intensive, and so we will instead label edge by two integers: the starting position of the first occurrence of $String(edge)$ in $Text$, denoted $Position(edge)$, and its length, denoted $Length(edge)$. For the modified suffix tree of $Text = "panamabananas\$"$ shown in the figure below, these two integers are colored blue and red, respectively. For example, the edge labeled "mabananas\$" from the original suffix tree is labeled by $Position(edge) = 4$ and $Length(edge) = 10$ in the figure below. There are two edges labeled "na" in the original suffix tree, and both of them are labeled by $Position(edge) = 2$ and $Length(edge) = 2$ in the figure below.

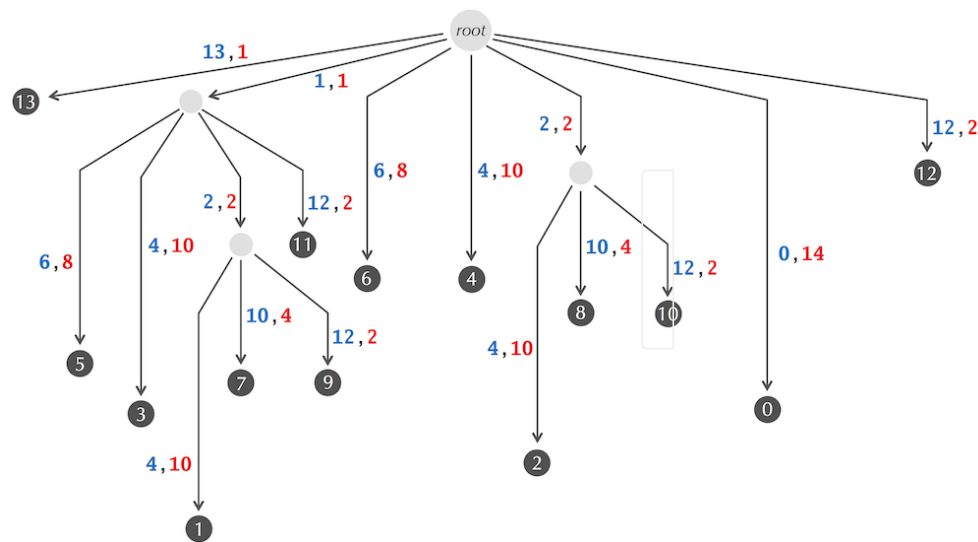


Figure: The modified suffix tree of *Text* = "panamabananas\$". For each edge, the initial position of the substring to which it corresponds in *Text* is shown in blue, and the length of this substring is shown in red.

The following pseudocode constructs a suffix tree using the modified suffix trie constructed by **MODIFIEDSUFFIXTRIECONSTRUCTION**. This algorithm will consolidate each non-branching path of the modified suffix trie into a single edge. For more on non-branching paths, recall CHARGING STATION: Maximal Non-Branching Paths in a Graph (<https://stepic.org/lesson/CS-Maximal-Non-Branching-Paths-in-a-Graph-6207>).

MODIFIEDSUFFIXTREECONSTRUCTION(*Text*)

Trie $\leftarrow$ **MODIFIEDSUFFIXTRIECONSTRUCTION**

for each non-branching path *Path* in *Trie*

 substitute *Path* by a single edge *edge* connecting the first and last nodes of *Path*

$Position(edge) \leftarrow Position(\text{first edge of } Path)$

$Length(edge) \leftarrow \text{number of edges of } Path$

return *Trie*

Return to Main Text [_ \(https://stepic.org/lesson/Suffix-Trees-296/step/3\)](https://stepic.org/lesson/Suffix-Trees-296/step/3)



A node v in a directed graph $Graph$ is called a **1-in-1-out node** if its indegree and outdegree are both equal to 1, i.e., $in(v) = out(v) = 1$. We can rephrase the definition of a "maximal non-branching path" from the main text as a path whose internal nodes are 1-in-1-out nodes and whose initial and final nodes are not 1-in-1-out nodes. Also, note that the definition from the main text does not handle the special case when $Graph$ has a connected component that is an **isolated cycle**, in which all nodes are 1-in-1-out nodes.

The **MAXIMALNONBRANCHINGPATHS** pseudocode below generates all non-branching paths in a graph. It iterates through all nodes of the graph that are not 1-in-1-out nodes and generates all non-branching paths starting at each such node. In a final step, **MAXIMALNONBRANCHINGPATHS** finds all isolated cycles in the graph.

MAXIMALNONBRANCHINGPATHS($Graph$)

$Paths \leftarrow$ empty list

for each node v in $Graph$

if v is not a 1-in-1-out node

if $out(v) > 0$

for each outgoing edge (v, w) from v

$NonBranchingPath \leftarrow$ the path consisting of the single edge (v, w)

while w is a 1-in-1-out node

 extend $NonBranchingPath$ by the outgoing edge (w, u) from w

$w \leftarrow u$

 add $NonBranchingPath$ to the set $Paths$

for each isolated cycle $Cycle$ in $Graph$

 add $Cycle$ to $Paths$

return $Paths$





CODE CHALLENGE: Implement **MAXIMALNONBRANCHINGPATHS**.

Input: The adjacency list of a graph whose nodes are integers.

Output: The collection of all maximal nonbranching paths in this graph.

Extra Dataset

(<http://bioinformaticsalgorithms.com/data/extradata/assembly/MaximalNonBranchingPaths.txt>)

Sample Input:

```
1 -> 2
2 -> 3
3 -> 4,5
6 -> 7
7 -> 6
```

Sample Output:

```
1 -> 2 -> 3
3 -> 4
3 -> 5
7 -> 6 -> 7
```

Time Limit: 5 mins

To solve this problem please visit
stepic.org/lesson/-6207/step/2



A naive approach for finding a longest shared substring of strings $Text_1$ and $Text_2$ would construct one suffix tree for $Text_1$ and another for $Text_2$. Instead, we will add "#" to $Text_1$, add "\$" to $Text_2$, and then constructing the single suffix tree for the concatenation of $Text_1$ and $Text_2$ (see the figure on the next step). We color a leaf in this suffix tree blue if it is labeled by the starting position of a suffix starting in $Text_1$; we color a leaf red if it is labeled by the starting position of a suffix starting in $Text_2$.

We also color the remaining nodes of the suffix tree blue, red, and purple according to the following rules:

- a node is colored blue or red if all leaves in its subtree (i.e., the subtree beneath it) are all blue or all red, respectively;
- a node is colored purple if its subtree contains both blue and red leaves.

We use $Color(v)$ to denote the color of node v .

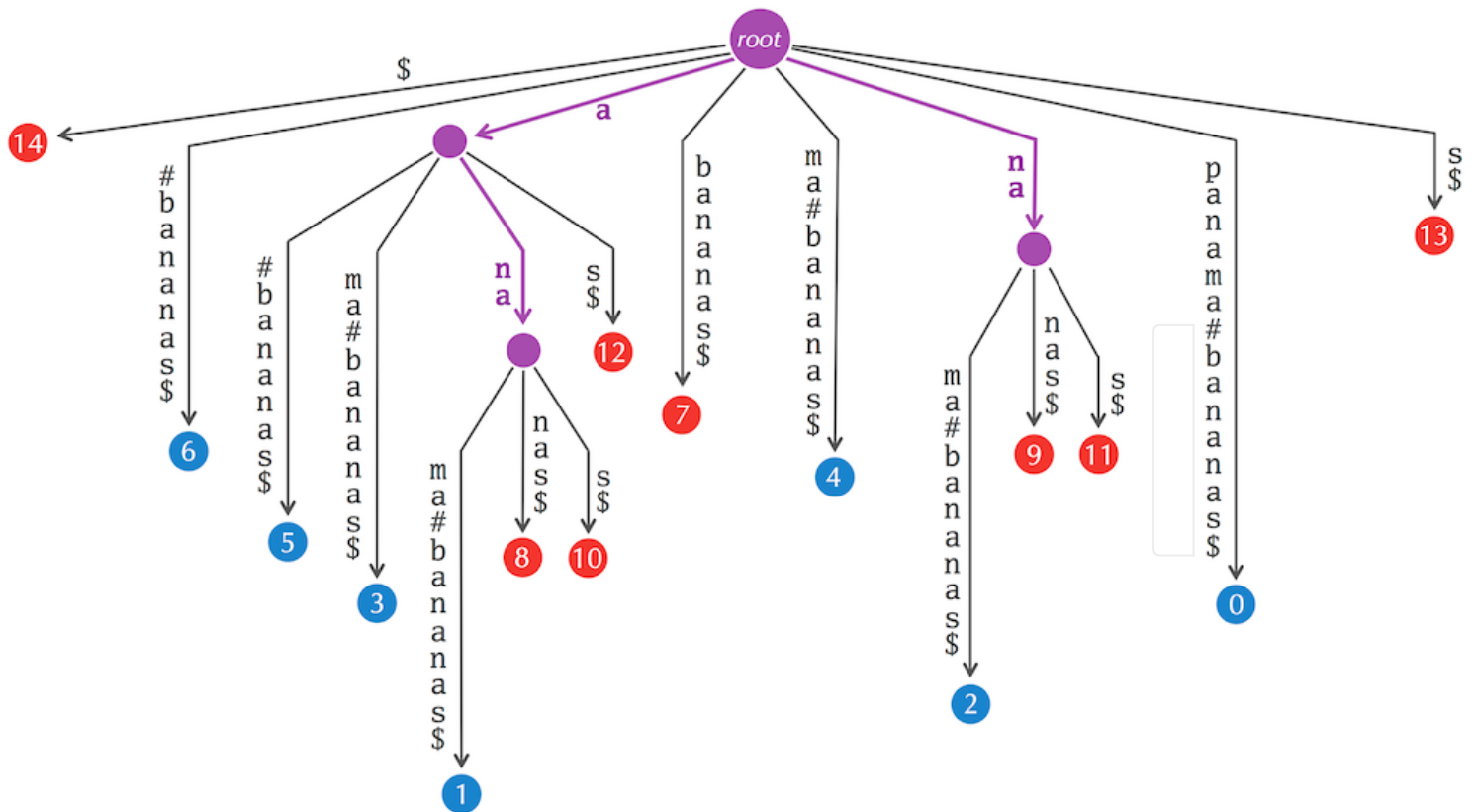




Figure: If $Text_1 = \text{"panama"}$ and $Text_2 = \text{"bananas"}$, then we can find a longest shared substring of these two strings by constructing $SuffixTree(\text{"panama\#bananas\$"})$, shown above. Leaves corresponding to suffixes starting in **"panama\#"** are colored blue; leaves corresponding to suffixes starting in **"bananas"** are colored red. Internal nodes whose subtrees have only blue or red leaves are colored blue or red, respectively; internal nodes whose subtrees have both red and blue leaves are colored purple. Every string of symbols spelled from the root to a purple node corresponds to a substring shared by $Text_1$ and $Text_2$.



There are three purple nodes in the figure from the previous step (reproduced below), and the strings spelled from the root to each of these nodes are "a", "ana", and "na". Note that these three substrings are shared by $Text_1 = \text{"panama"}$ and $Text_2 = \text{"bananas"}$. This is no accident, as illustrated by the following two (ungraded) exercises.



EXERCISE BREAK: Prove that each purple node in the suffix tree for $Text_1$ and $Text_2$ corresponds to a substring shared by $Text_1$ and $Text_2$.

EXERCISE BREAK: Prove that each blue (respectively, red) node in the suffix tree for $Text_1$ and $Text_2$ corresponds to a substring that appears in $Text_1$ but not in $Text_2$ (respectively, $Text_2$ but not in $Text_1$).



The previous two exercises imply that in order to find the longest shared substring between $Text_1$ and $Text_2$, we need to examine all purple nodes as well as the strings spelled by paths leading to the purple nodes. A longest such string yields a solution to the Longest Shared Substring Problem.

TREECOLORING, whose pseudocode is shown below and illustrated on the next step, colors the nodes of a suffix tree from the leaves upward. This algorithm assumes that the leaves of the suffix tree have been labeled "blue" or "red" and all other nodes have been labeled "gray". A node in a tree is called **ripe** if it is gray but has no gray children.

```
TREECOLORING(ColoredTree)  
  while ColoredTree has ripe nodes  
    for each ripe node  $v$  in ColoredTree  
      if there exist differently colored children of  $v$   
         $Color(v) \leftarrow \text{"purple"}$   
      else  
         $Color(v) \leftarrow \text{color of all children of } v$   
  return ColoredTree
```



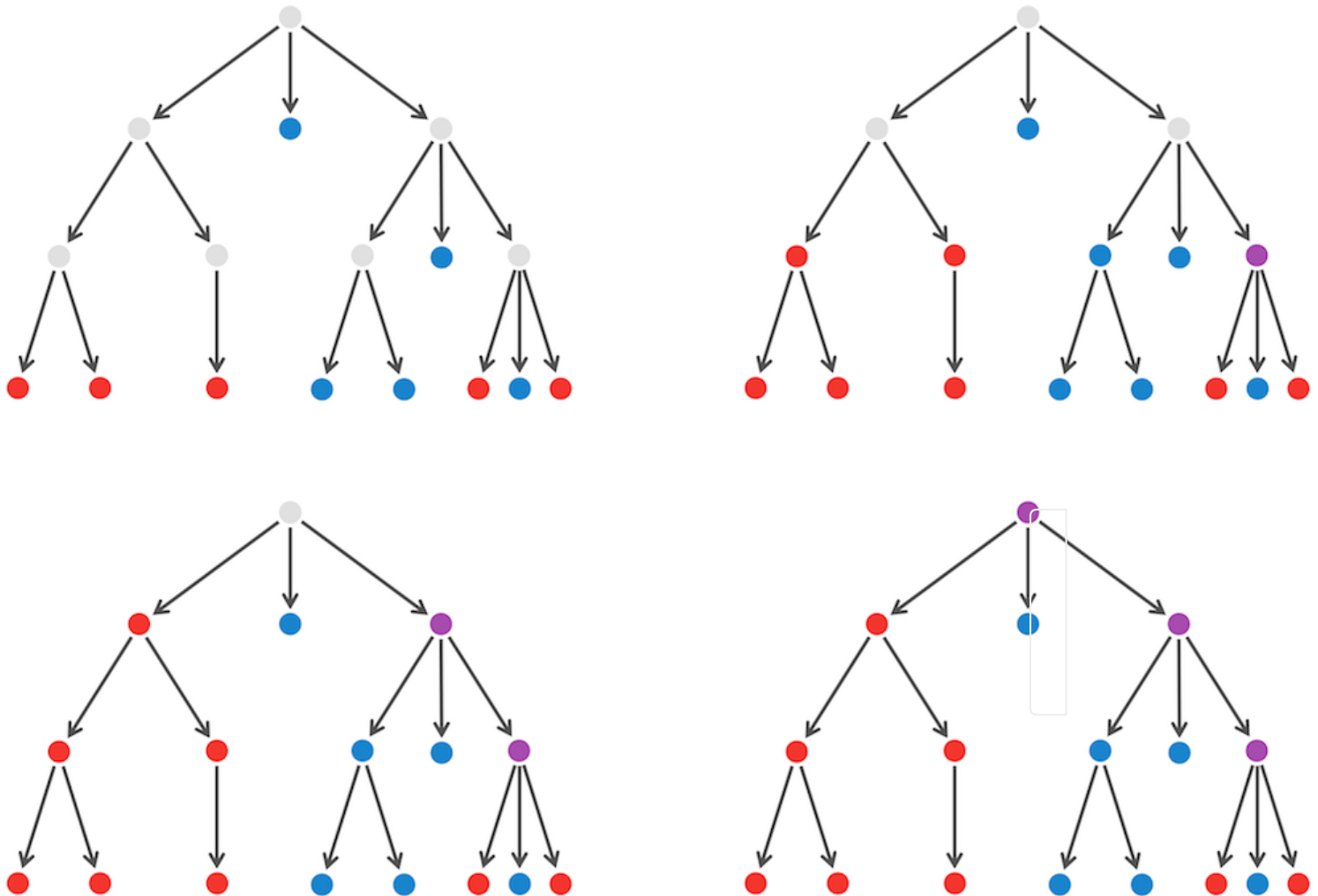


Figure: An illustration of the steps taken by **TREECOLORING** for a tree with an initial coloring of leaves as red or blue (top left).



Tree Coloring Problem: *Color the internal nodes of a tree given the colors of its leaves.*

Input: An adjacency list, followed by color labels for leaf nodes.

Input: Color labels for all nodes, in any order.

CODE CHALLENGE: Solve the Tree Coloring Problem.

Extra Dataset

(<http://bioinformaticsalgorithms.com/data/extradata/bowtie/TreeColoring.txt>)

Sample Input:

```
0 -> {}
1 -> {}
2 -> 0,1
3 -> {}
4 -> {}
5 -> 3,2
6 -> {}
7 -> 4,5,6
-
0: red
1: red
3: blue
4: blue
6: red
```

Sample Output:

```
0: red
1: red
2: red
3: blue
4: blue
5: purple
6: red
7: purple
```


Time Limit: 5 mins

To solve this problem please visit
stepic.org/lesson/-9665/step/6



When genomes are assembled from DNA taken from a number of donors, the reference genome is a mosaic of donor genomes. The existing reference human genome was derived from thirteen volunteers in the United States; it continues to be improved by fixing errors and filling in the remaining gaps (there are currently over a hundred).

The reference genome is often used as a template on which new individual genomes can be rapidly assembled. In regions with high diversity, the reference genome may differ significantly from individual genomes. An example of a high-diversity region of the human genome is the **major histocompatibility complex**, a family of genes powering vertebrate immune systems. Since the immune system is very diverse, these genes have an unusually large number of alternate forms. As a result, two individuals hardly ever have exactly the same set of genes from this complex. Comparison of the reference genome and individual human genomes typically reveals about three million SNP differences, and about 1% of an individual human genome cannot be matched to the reference genome at all.

Return to Main Text [_ \(https://stepic.org/lesson/What-Causes-Ohdo-Syndrome-292/step/2\)](https://stepic.org/lesson/What-Causes-Ohdo-Syndrome-292/step/2)



Until recently, biologists focused primarily on SNPs in the human genome, assuming that rearrangements and indels are relatively rare. In 2005, Evan Eichler surprised biologists when he found hundreds of rearrangements and indels separating the genomes of two individuals. This finding was important because rearrangements and indels are often hallmarks of disease; for example, repeated insertions of the nucleotide triplet CAG increases the severity of Huntington's disease.

In 2013, Gerton Lunter revealed the true extent of indels in the human population by identifying over a *million* indels in a cohort of over a hundred individuals. Intriguingly, he found that over half of the indels occurred in just 4% of the genome; in other words, some regions of the human genome represent "indel hotspots". As the catalog of human rearrangements and indels grows, biologists are gaining the ability to identify frequently mutated genes as well as implicate rearrangements and indels when diagnosing complex disorders.

Return to Main Text [_https://stepic.org/lesson/What-Causes-Ohdo-Syndrome-292/step/2_](https://stepic.org/lesson/What-Causes-Ohdo-Syndrome-292/step/2)



The **Aho-Corasick algorithm** for the Multiple Pattern Matching Problem was invented by Alfred Aho and Margaret Corasick in 1975. The running time of their algorithm is $O(|Patterns| + |Text| + m)$, where m is the number of output matches.

The diagram shows a search tree starting from a 'Root' node. The root has four children: 'a', 'b', 'n', and 'p'. The 'b' child is highlighted in blue. This blue path continues through nodes labeled 'a', 'n', 'd', 'a', 'n', 'a', ending at a final node. From the 'd' node in this path, there are two red branches leading to 'a' and 'n', which then lead to 'n' and 'a' respectively, completing the word 'bandana'. Other branches from the root ('a', 'n', 'p') and further down the tree are shown in gray, representing other potential paths or words.



However, we have thrown away some important information: we *already matched* the first two symbols of "antenna" when searching through "bantenna". Thus, a more sensible strategy is to jump directly to the node "an" and then continue downward in the trie. Indeed, this strategy will eventually match the pattern "antenna". We can implement this "jumping ahead" strategy by augmenting the tree with a **failure edge** connecting node "ban" to node "an" (shown in the figure below). More generally, failure edges are formed by connecting node v to node w if w is the longest suffix of v that appears in the trie. The Aho-Corasick algorithm follows failure edges whenever a mismatch is found in order to avoid going all the way back to the root, thus saving time.

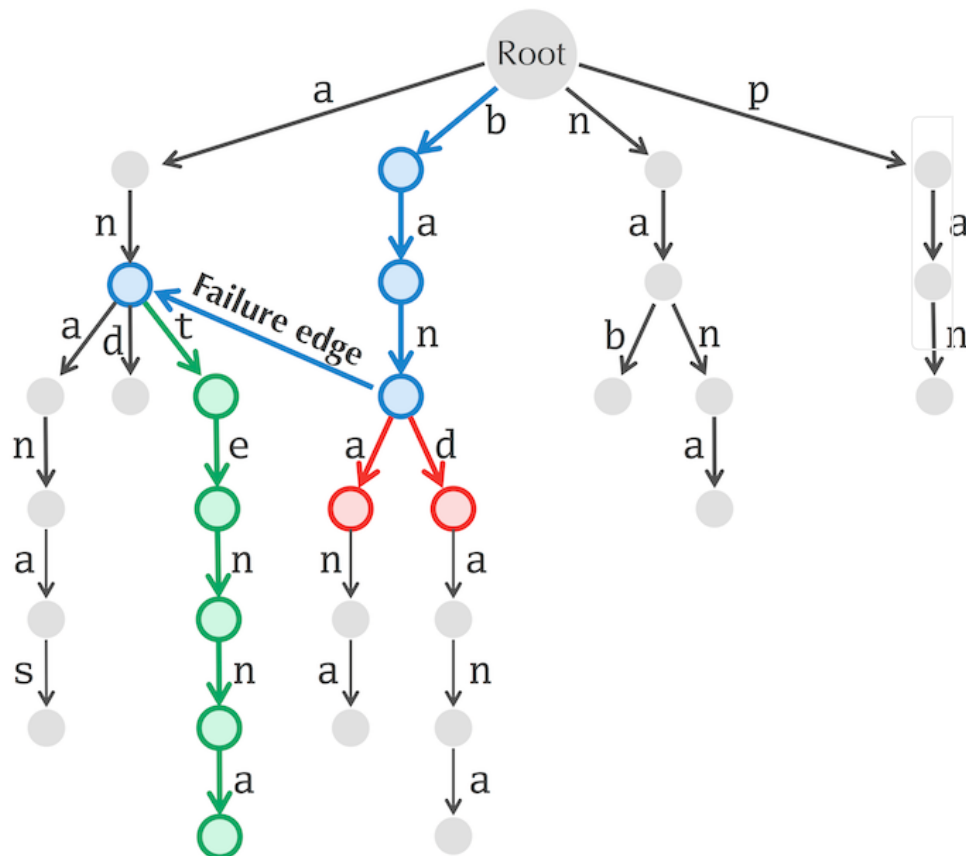


Figure: The trie considered in the main text with an additional failure edge. The failure edge allows us to jump straight from the path beginning with "ban" to the path beginning with "an".

